# Horizontal Privilege Escalation & IDOR Master Cheatsheet

A comprehensive reference guide for discovering, analyzing, and securing Insecure Direct Object References & BOLA flaws.

**OWASP A01:2021**

## 1 Core Concept & Root Cause

**Horizontal Privilege Escalation (BOLA / IDOR):** Occurs when a user (User A) gains unauthorized access to resources or modifies data belonging to another user (User B) possessing the *same privilege level* by tampering with a **Direct Object Identifier**.

**Root Cause:** The server blindly trusts incoming object parameters (e.g., `user_id=102`) supplied by the user client without verifying **Session Ownership** on the server side.

## 2 Identifier Patterns & Categories

| Identifier Type | Example Parameter | Target Pattern & Behavior |
|---|---|---|
| **Sequential Numeric IDs** | `id=1001`, `account_id=554` | Predictable auto-incrementing integers easily enumerated via fuzzing. |
| **Hashed / Encoded IDs** | `user=c4ca4238a0b92382...` | Simple hashing algorithms (e.g., MD5/Base64 of ID '1') easy to decode. |
| **Static Strings** | `user=john_doe`, `email=victim@target.com` | Direct reliance on usernames or email addresses as keys. |
| **UUID / GUID References** | `id=a1b2c3d4-e5f6-7890-abcd...` | Complex randomized IDs; check for leaks in response bodies or JS files. |

## 3 Discovery & Attack Methodology

1. **Parameter Enumeration:** Browse target application functions while proxying traffic through Burp Suite to isolate parameter identifiers (e.g., `account_id`, `doc_id`, `profile_id`).
2. **Multi-Account Setup:** Provision two distinct peer accounts with identical privileges (Attacker Session & Victim Session) for clear isolation.
3. **Context & Token Swapping:** Intercept Account A's request, replace the object parameter with Account B's identifier, but keep Account A's session cookies/tokens intact.
4. **Response Analysis:** Verify unauthorized access if an `HTTP 200 OK` response with victim data is returned instead of `403 Forbidden`.

## 4 Advanced Bypass Vectors

### A. HTTP Method Switching

If `GET /api/user/102` returns 403, try `POST`, `PUT`, `DELETE`, or send `X-HTTP-Method-Override: GET`.

### B. HTTP Parameter Pollution (HPP)

Supply duplicate keys such as `?user_id=MY_ID&user_id=VICTIM_ID` to test backend parser priority.

### C. Data Format Wrapping (JSON / Array)

Convert scalar IDs into arrays `{"id": [101, 102]}` or `id[]=102` to bypass primitive input validation filters.

### D. GraphQL & Mass Assignment

Modify object IDs within GraphQL mutations or inject unlisted fields like `"owner_id": 102` into JSON update payloads.

## 5 Real-World Attack Scenarios

**Vulnerable GET Request Scenario:**

```
GET /api/v1/account/balance?account_id=54821 HTTP/1.1 Host: target-app.com Cookie: session_token=ATTACKER_SESSION_COOKIE User-Agent: Mozilla/5.0 (Security Tester)
```

**Vulnerable POST / JSON Body Request Scenario:**

```
POST /api/v1/user/update_profile HTTP/1.1 Host: target-app.com Authorization: Bearer ATTACKER_JWT_TOKEN Content-Type: application/json { "user_id": 1042, "email": "victim_user@example.com", "phone": "+123456789" }
```

## 6 Automation Tools & Burp Extensions

### Burp Suite - Autorize:

Automates authorization testing by repeating requests with low-privileged cookies and identifying IDORs in real-time.

### Param Miner & AutoRepeater:

Discovers unlinked hidden parameters and automatically swaps predefined values across background requests.

## 7 Secure Code Remediation

❌ **Vulnerable Code (Spring Boot Controller - IDOR):**

```
@GetMapping("/api/invoice") public Invoice getInvoice(@RequestParam String invoiceId) { // Server fetches resource directly without validating session ownership return invoiceRepository.findById(invoiceId); }
```

✅ **Secured Code (Session-Based Access Check):**

```
@GetMapping("/api/invoice") public Invoice getInvoice(@RequestParam String invoiceId, HttpSession session) { User currentUser = (User) session.getAttribute("currentUser"); Invoice invoice = invoiceRepository.findById(invoiceId); // Enforce strict ownership authorization check if (invoice == null || !invoice.getOwnerId().equals(currentUser.getId())) { throw new AccessDeniedException("403 Forbidden: Unauthorized Access"); } return invoice; }
```

## 8 Interactive Verification Checklist

- [ ] Tested object ID parameter swapping across `GET`, `POST`, `PUT`, and `DELETE` endpoints.
- [ ] Inspected hidden JS files and response bodies for leaked UUIDs/GUIDs.
- [ ] Tested data structure mutation (e.g., converting IDs to JSON arrays or nested objects).
- [ ] Verified server-side ownership verification based on trusted session identifiers.